

Using ilike

In PostgreSQL, it is possible to perform a case-insensitive like query by using the `ilike` operator. In this situation, our query would become the following:

```
forumdb=> select * from categories where description ilike '%DISCUSS%';
 pk | title | description
----+-----+-----
  1 | Database | Database related discussions
  2 | Unix | Unix and Linux discussions
(2 rows)
```

This is the PostgreSQL way of solving the case-insensitive like query issue that we encountered previously.

Using distinct

We will now discuss another kind of query: the `distinct` query. Firstly, however, we need to introduce another very useful function for the DBA called the `coalesce` function. The `coalesce` function, given two or more parameters, returns the first value that is not `NULL`.

For example, let's use the `coalesce` function for the test value:

```
forumdb=> select coalesce(NULL,'test');
 coalesce
-----
 test
(1 row)
```

In the preceding query, the `coalesce` function returns `test` because the first argument is `NULL` and the second argument is not `NULL`.

Now, let's insert a new category:

```
forumdb=> insert into categories (title) values ('New Category');
INSERT 0 1
```

And then let's perform the following query:

```
forumdb=# \pset null (NULL)
Null display is "(NULL)".
```

```

forumdb=> select pk,title,description from categories;
 pk |          title          |          description
----+-----+-----
  1 | Database                 | Database related discussions
  2 | Unix                     | Unix and Linux discussions
  3 | Programming Languages    | All about programming languages
  4 | New Category             | (NULL)
(4 rows)

```

In the example above, the field description has a NULL value for the title New Category.

Now let's try to use the coalesce function to show the value No Description instead of NULL.

```

forumdb=> select pk,title,coalesce(description,'No description') from
categories;
 pk |          title          |          coalesce
----+-----+-----
  1 | Database                 | Database related discussions
  2 | Unix                     | Unix and Linux discussions
  3 | Programming Languages    | All about programming languages
  4 | New Category             | No description
(4 rows)

```

In the preceding code, the coalesce function transforms any NULL value into the string No description. Another thing that isn't very user-friendly about the coalesce function is that the name of the field that is given when a function is called is not the name we would want for our query. In this case, the second field of the resultset is called coalesce, which is not the name we would prefer; this is because if we are working in a team, a human-readable name is preferred.

In PostgreSQL, an alias can be assigned to any field in a query. For example, we can assign an alias to the coalesce field as follows:

```

forumdb=> select pk,title,coalesce(description,'No description') as
description from categories;
 pk |          title          |          description
----+-----+-----
  1 | Database                 | Database related discussions
  2 | Unix                     | Unix and Linux discussions
  3 | Programming Languages    | All about programming languages

```

```

4 | New Category          | No description
(4 rows)

```

Now the resultset has the description field instead of the coalesce field.

If we want to use an alias with spaces or capital letters, we have to quote the alias using "", as in the following example:

```

forumdb=> select pk,title,coalesce(description,'No description') as
"Description" from categories;
pk | title | Description
----+-----+-----
1 | Database | Database related discussions
2 | Unix | Unix and Linux discussions
3 | Programming Languages | All about programming languages
4 | New Category | No description
(4 rows)

```

The resultset doesn't have an alias of Description (uppercase) but does have an alias of description (lowercase), which doesn't seem right. Now let's insert another record like this:

```

forumdb=> insert into categories (title,description) values
('Database','PostgreSQL');
INSERT 0 1

```

And let's perform this query:

```

forumdb=> select title from categories order by title;
title
-----
Database
Database
New Category
Programming Languages
Unix
(5 rows)

```

As we can see in the query above, there are 2 records with the same value, Database; if we want to show all the distinct values, we have to use the DISTINCT clause:

```

forumdb=> select distinct title from categories order by title;
title

```

```

-----
Database
New Category
Programming Languages
Unix
(4 rows)

```

In the preceding query, we have used the `select distinct` statement. The `select distinct` statement is used to return only distinct (different) values. Internally, the `distinct` statement involves a data sort for large tables, which means that if a query uses the `distinct` statement, the query may become slower as the number of records increases.

Using limit and offset

The `limit` clause is the PostgreSQL way to limit the number of rows returned by a query, whereas the `offset` clause is used to skip a specific number of rows returned by the query.

`limit` and `offset` are used to return a portion of data from a resultset generated by a query; the `limit` clause is used to limit the number of records in output and the `offset` clause is used to provide PostgreSQL with the position in the resultset from which to start returning data.

They can be used independently or together.

Now let's test `limit` and `offset` using the following queries:

```

forumdb=> select * from categories order by pk limit 1;
 pk | title | description
-----+-----+-----
  1 | Database | Database related discussions
(1 row)

```

The preceding query returns only the first record that we have inserted; this is because the `pk` field is an integer type with a default value generated always as the identity.

If we want to see the first two records that were inserted, we have to perform the following query:

```

forumdb=> select * from categories order by pk limit 2;
 pk | title | description
-----+-----+-----
  1 | Database | Database related discussions
  2 | Unix | Unix and Linux discussions
(2 rows)

```

If we only want the second record that was inserted, we have to perform the following query:

```
forumdb=> select * from categories order by pk offset 1 limit 1;
pk | title | description
-----+-----+-----
 2 | Unix  | Unix and Linux discussions
(1 row)
```

`offset` and `limit` are very useful when we want to return data in a paged way.

Another valuable function of `limit` is that it can create a new table from an existing table. For example, if we want to create a table called `new_categories` starting from the `categories` table, we have to execute the following statement:

```
forumdb=> create table new_categories as select * from categories limit 0;
SELECT 0
```

This statement will copy into the `new_categories` table only the data structure of the table `categories`.

The `SELECT 0` clause means that no data has been copied into the `new_categories` table; only the data structure has been replicated, as we can see here:

```
forumdb=> \d new_categories
          Table "forum.new_categories"
  Column   | Type   | Collation | Nullable | Default
-----+-----+-----+-----+-----
pk         | integer |           |          |
title      | text    |           |          |
description | text    |           |          |
```

Using subqueries

In this section, we will talk about subqueries. Subqueries can be described as nested queries – we can nest a query inside another query using parentheses. Subqueries can return a single value or a recordset, just like regular queries. We will start by introducing subqueries using the `IN/NOT IN` operator.

Subqueries and the IN/NOT IN condition

Let's start with the IN operator; we can use the IN operator inside a where clause instead of using multiple OR conditions. For example, if you wanted to search for all categories that have the value pk=1 or the value pk=2, you would have to perform the following statement:

```
forumdb=> select * from categories where pk=1 or pk=2;
pk | title | description
----+-----+-----
  1 | Database | Database related discussions
  2 | Unix | Unix and Linux discussions
(2 rows)
```

Another way to reach the same outcome is the following:

```
forumdb=> select * from categories where pk in (1,2);
pk | title | description
----+-----+-----
  1 | Database | Database related discussions
  2 | Unix | Unix and Linux discussions
(2 rows)
```

An operator similar to the IN operator but with reverse functionality is the NOT IN operator. For example, if we wanted to search for all categories that do not have pk=1 or pk=2, we would have to execute the following:

```
forumdb=> select * from categories where pk not in (1,2);
pk | title | description
----+-----+-----
  3 | Programming Languages | All about programming languages
  4 | New Category | (NULL)
  5 | Database | PostgreSQL
(3 rows)
```

Now, we can insert some data into users and the posts table:

```
forumdb=> insert into users (username,email) values ('luca_ferrari','luca@pgtraining.com'),('enrico_pirozzi','enrico@pgtraining.com');
INSERT 0 2
```

```

forumbd=> insert into posts (title,content,author,category) values
('Indexing PostgreSQL','Btree in PostgreSQL is....',1,1);
INSERT 0 1
forumbd=> insert into posts (title,content,author,category) values
('Indexing Mysql','Btree in Mysql is....',1,1);
INSERT 0 1
forumbd=> insert into posts (title,content,author,category) values ('Data
types in C++','Data type in C++ are ..' ,2,3);
INSERT 0 1

```

The records present in the posts table are now as follows:

```

forumbd=> \x
Expanded display is on.
forumbd=> select pk,title,content,author,category from posts;
-[ RECORD 1 ]-----
pk          | 1
title       | Indexing PostgreSQL
content     | Btree in PostgreSQL is....
author      | 1
category    | 1
-[ RECORD 2 ]-----
pk          | 2
title       | Indexing Mysql
content     | Btree in Mysql is....
author      | 1
category    | 1
-[ RECORD 3 ]-----
pk          | 3
title       | Data types in C++
content     | Data type in C++ are ..
author      | 2
category    | 3

```

Suppose we now want to search for all posts that belong to the Database category. To do this, we can use several methods.

The following method uses subqueries:

```

forumdb=> select pk,title,content,author,category from posts where
category in (select pk from categories where title ='Database');
-[ RECORD 1 ]-----
pk          | 1
title       | Indexing PostgreSQL
content     | Btree in PostgreSQL is....
author      | 1
category    | 1
-[ RECORD 2 ]-----
pk          | 2
title       | Indexing Mysql
content     | Btree in Mysql is....
author      | 1
category    | 1

```

The subquery is represented by the following:

```

forumdb=> \x
Expanded display is off.

forumdb=> select pk from categories where title ='Database';
 pk
----
  1
  5
(2 rows)

```

This statement extracts the values pk=1 and pk=5 from the category table and the external query searches the records in the posts table that have pk=1 or pk=5. Similarly, if you wanted to search for all post values that do not belong to the Database category, you would have to perform the following statement:

```

forumdb=> \x
Expanded display is on.

forumdb=> select pk,title,content,author,category from posts where
category not in (select pk from categories where title ='Database');

```

```
-[ RECORD 1 ]-----
pk          | 3
title       | Data types in C++
content     | Data type in C++ are ..
author      | 2
category    | 3
```

Subqueries and the EXISTS/NOT EXISTS condition

The EXISTS statement is used when we want to check whether a subquery returns (TRUE), and the NOT EXISTS statement is used when we want to check whether a subquery does not return (FALSE). For example, if we wanted to write the same conditions written previously using the EXISTS/NOT EXISTS condition, we'd have to perform the following:

```
forumdb=> select pk,title,content,author,category from posts where exists
(select 1 from categories where title ='Database' and posts.category=pk);
-[ RECORD 1 ]-----
pk          | 1
title       | Indexing PostgreSQL
content     | Btree in PostgreSQL is....
author      | 1
category    | 1
-[ RECORD 2 ]-----
pk          | 2
title       | Indexing Mysql
content     | Btree in Mysql is....
author      | 1
category    | 1
```

The preceding query returns the same results as the query written with the IN condition.

Similarly, if we wanted to search for all post values that do not belong to the Database category using the NOT EXISTS condition, we'd have to write the following:

```
forumdb=> select pk,title,content,author,category from posts where not
exists (select 1 from categories where title ='Database' and posts.
category=pk);
-[ RECORD 1 ]-----
pk          | 3
title       | Data types in C++
```

content		Data type in C++ are ..
author		2
category		3

Both queries written with the IN condition and with the EXISTS condition are called **semi-join queries**, and we will be looking at joins in the next section.

Learning about joins

Let’s address what a join is, how many types of joins exist, and what they are used for. We can think of a join as a combination of rows from two or more tables.

For example, the following query returns all the combinations from the rows of the category table and the rows of the posts table:

```
forumdb=> select c.pk,c.title,p.pk,p.category,p.title from categories
c,posts p;
```

pk	title	pk	category	title
1	Database	1	1	Indexing PostgreSQL
2	Unix	1	1	Indexing PostgreSQL
3	Programming Languages	1	1	Indexing PostgreSQL
4	New Category	1	1	Indexing PostgreSQL
5	Database	1	1	Indexing PostgreSQL
1	Database	2	1	Indexing Mysql
2	Unix	2	1	Indexing Mysql
3	Programming Languages	2	1	Indexing Mysql
4	New Category	2	1	Indexing Mysql
5	Database	2	1	Indexing Mysql
1	Database	3	3	Data types in C++
2	Unix	3	3	Data types in C++
3	Programming Languages	3	3	Data types in C++
4	New Category	3	3	Data types in C++
5	Database	3	3	Data types in C++

(15 rows)